Assignment 2: GAN and Diffusion Model

Instructions:

- Follow the instructions of each question carefully.
- Submit each section as a separate **jupyter notebook** that shows outputs of each stage.
- **If you cannot explain any part of the code, you will be given zero for that part of the assignment.**

Section 1: Implement a Generative Adversarial Network (GAN) with Modifiability for Conditional GAN (cGAN) (100 marks)

Your task is to implement a **Generative Adversarial Network (GAN) from scratch** using PyTorch. This GAN will be trained on the **MNIST dataset** to generate realistic handwritten digits. The design should be flexible enough so that it can be extended into a **Conditional GAN (cGAN)** with minimal modifications.

1. Architecture Details:

- **Generator**
  - Input: **Random noise vector** (**latent space** of size **100**).
  - Fully connected layers with **LeakyReLU** activations.
  - Output reshaped into a **28×28 grayscale image**.
  - Use **Tanh activation** in the final layer.
  - **[Modification for cGAN]**: Ensure that the generator can accept **both noise and a label** as input.
- **Discriminator**
  - Input: **28×28 grayscale image**.
  - Fully connected layers with **LeakyReLU** activations.
  - Outputs a **single value (real or fake)** using **sigmoid activation**.
  - **[Modification for cGAN]**: The discriminator should be able to **process both the image and a label** as input.
- **Hyperparameters:**
  - **Learning Rate**: 0.0002
  - **Optimizer**: Adam ($\beta1=0.5$, $\beta2=0.999$)
  - **Batch Size**: 128
  - **Epochs**: 50
  - **Loss Function**: Binary Cross-Entropy (BCE)
  - **Latent Space Dimension**: 100

2. **Tasks:**

1. Create a new notebook GAN.ipynb
2. **Implement the Generator and Discriminator networks.**
3. **Define the training loop for adversarial training:**
   - Train the **Discriminator** using both real and generated (fake) images.
   - Train the **Generator** to **fool** the Discriminator.
4. **Train the model on the MNIST dataset using the given hyperparameters.**
5. **Generate and visualize images after every 10 epochs to track GAN's progress.**
6. **Save the trained Generator model and use it for inference to generate new digits.**
7. **Modify the GAN to a Conditional GAN (cGAN)**:

- Modify the **Generator** to accept both a **random noise vector** and **a class label** as input.
- Modify the **Discriminator** to accept both an **image** and **its corresponding label** as input.
- Train the cGAN model and compare the quality of its generated images against the vanilla GAN.

3. Hints for Conditional GAN (cGAN) Implementation:

- Use **one-hot encoding** for labels and concatenate them with the noise vector in the **Generator**.
- For the **Discriminator**, concatenate labels with the flattened image input or inject the label through **embedding layers**.
- Consider **using an auxiliary loss** to help guide the Generator in generating images of the correct class.

Section 2: Wasserstein GAN (WGAN) Implementation (50 Marks)

1. Extend your implementation in Section 1 to support Wasserstein GAN. Conceptual changes required:

- Replace Discriminator with a **Critic** C
- Remove Sigmoid activation.
- Output raw scalar values.
- Replace Binary Cross Entropy loss with Wasserstein loss.
- Train Critic 5 times per 1 Generator update.
- Apply **weight clipping** to Critic parameters. Weight clipping range: [-0.01, 0.01]

2. Loss Functions

Critic Loss: $L_C = -\mathbb{E}[C(x)] + \mathbb{E}\big[C\big(G(z)\big)\big]$ Generator Loss: $L_G = -\mathbb{E}[C(G(z))]$

3. Hyperparameter Adjustments

- Learning rate: 0.00005
- Optimizer: RMSprop

4. Tasks:

1. Implement WGAN training loop.
2. Train on MNIST.
3. Visualize generated samples every 10 epochs.
4. Plot Wasserstein loss curves.
5. Compare stability against Vanilla GAN.

Section 3: Denoising Diffusion Probabilistic Model (100 marks)

In this task, you will implement a **Denoising Diffusion Probabilistic Model (DDPM)** to generate images from noise. This model follows a **forward diffusion process** (adding noise) and a **reverse process** (removing noise using a trained neural network). You will work with the **MNIST dataset** (handwritten digits) and use a lightweight **U-Net-based model** for denoising.

Dataset: MNIST

- The dataset consists of **grayscale (1-channel) images** of digits **0-9**.
- Each image has a size of **28x28 pixels**.
- You will use **PyTorch's torchvision** to load the dataset.

Tasks:

1. Implement the Diffusion Process

a. Forward Process (Adding Noise)

- Define a **noise schedule** of length **T = 1000** using a **linear schedule**:
  - The noise level (β_t) increases gradually from **1e-4** to **0.02**.
- Implement a function to **add Gaussian noise** to an image at a given timestep t.

b. Reverse Process (Denoising)

- Train a **lightweight U-Net model** that predicts the **added noise** at each step.
- The model should take:
  - A noisy image.
  - The current timestep **t** (as an additional input).
  - Return the predicted noise.

2. Implement the Neural Network

Model Architecture:

- A simplified **U-Net-based model**:
  - **Encoder:**
    - 3 convolutional layers with increasing channels **(32 → 64 → 128)**.
    - Each layer reduces the spatial size using a **stride of 2**.
  - **Middle block:**
    - A single convolutional layer with **ReLU activation**.
  - **Decoder:**
    - 3 transposed convolution layers to upsample the feature maps.
    - Final **1-channel output** for noise prediction.
- The model should **concatenate the timestep t** with the input image.

3. Training and Evaluation

Hyperparameters:

Parameter	Value
Timesteps (T)	1000
Batch Size	64
Image Size	28x28
Learning Rate	1e-3
Optimizer	Adam
Loss Function	Mean Squared Error (MSE)
Epochs	10

4. Training Steps

- Sample a batch of images from MNIST.
- Select random timesteps t for each image.
- Generate noisy images using the **forward diffusion process**.
- Train the model to predict the noise using **MSE loss**.
- Update weights using **Adam optimizer**.

5. Image Generation

After training, implement the **sampling procedure**:

- Start with **pure Gaussian noise**.
- Iteratively apply the **reverse denoising process** for **T steps**.
- Output the final **denoised image**.